

# *Számítási és tárolási felhők*

## *(3. ea)*

*Üzleti PaaS szolgáltatások*

Szeberényi Imre, Ludmány Balázs,  
BME IIT

<szebi@iit.bme.hu>



MŰEGYETEM 1782

# *PaaS rendszerek célkitűzése*

- Egyszerű környezetet adjanak (web) alkalmazások fejlesztésére és futtatására.
- A futtatást forgalom-függetlenül támogassák.
- A fejlesztőnek csak az alkalmazás elkészítésére kelljen koncentrálni.
- Automatikusan támogassák a skálázást.
- Serverless futtatás támogatása

# *Platform szolgáltatások*



- Alkalmazás és szolgáltatás konténerek
- Konténer-orkesztráció (Kubernetes)
  - skálázás, önjavítás, frissítés menedzsment
- BP szolgáltatások
- Integrációs szolgáltatások
- Alkalmazásszervező szolgáltatások
- Alkalmazás életciklus menedzsment

# *Platform szolgáltatások /2*

---

- Eseményfeldolgozás
- Funkciók futtatása idő- és eseményalapú triggerrel (FaaS)
- Átmeneti és perzisztens tárolás
- Registry és repository
- Azonosítás
- Biztonság

# *Programozási modell példák*

- Actor model (message passing)
- RESTful interactions
- Dynamic recoverability
- Consensus protocols
- Asynchronous interactions
- Shared nothing and sharding
- Service orchestration
- MapReduce
- Event-driven komponensek

# AppEngine



- Google PaaS szolgáltatása 2008-tól
- Serverless platform
- Támogatott nyelvek:
  - Java, Python, PHP, Go, Ruby, .NET, ...
- Adatbázis:
  - App Engine Datastore/Cloud Firestore (NoSQL)
  - Google Cloud SQL
  - Google Cloud Storage (REST)

# *AppEngine / 2*

- Egyéb szolgáltatások:
  - URL fetch
  - Mail
  - Memcache
- Biztonság (Sandbox)
  - Internetről csak HTTP/HTTPS
  - Nincs lokális fájl írás
  - Válaszidő 60 sec
  - Válasz után nincs fork

# *AppEngine / 3*

- Használatát mikor ajánlja a Google?
  - Nem kell servert telepíteni, karbantartani
  - Végtelen skálázhatóság
  - Előre nehezen becsülhető terhelés
  - A feladat darabolható (60s)
  - Nem kell lokális fájlrendszer

<https://console.developers.google.com/start/appengine>

<https://cloud.google.com/free-trial/>



# *AppScale*

- AppScale szolgáltatása 2009-től
- Apache licence
- Támogatott nyelvek:
  - Java, Python, PHP, Go
- Támogatott platformok:
  - Google Compute Engine
  - Amazon (EC2)
  - Softlayer (IBM)
  - MS Azure
  - RackSpace
  - DigitalOcean
  - OpenStack
  - CloudStack
  - Eucalyptus
  - KVM
  - Xen

# *AppScale APIs*



- Data management
  - MySQL, Cassandra, HBase, Hypertable, AWS SimpleDB, Azure SQL, Bigtable
  - Distributed transaction support, Memcache, blobstore
- Users
  - Messaging, email, authentication
- Tasking
  - Cron, task queues
- Web access
  - Fetch, export, user interaction

# *AppScale APIs /2*

---

- Computation intensive
  - Parallel/concurrency support
    - MPI, X10
  - Distributed simulation (biochemistry)
- Data analytics
  - Map-reduce (hadoop, pig, hive)
  - R, Matlab
  - Yahoo! S4 (streaming)

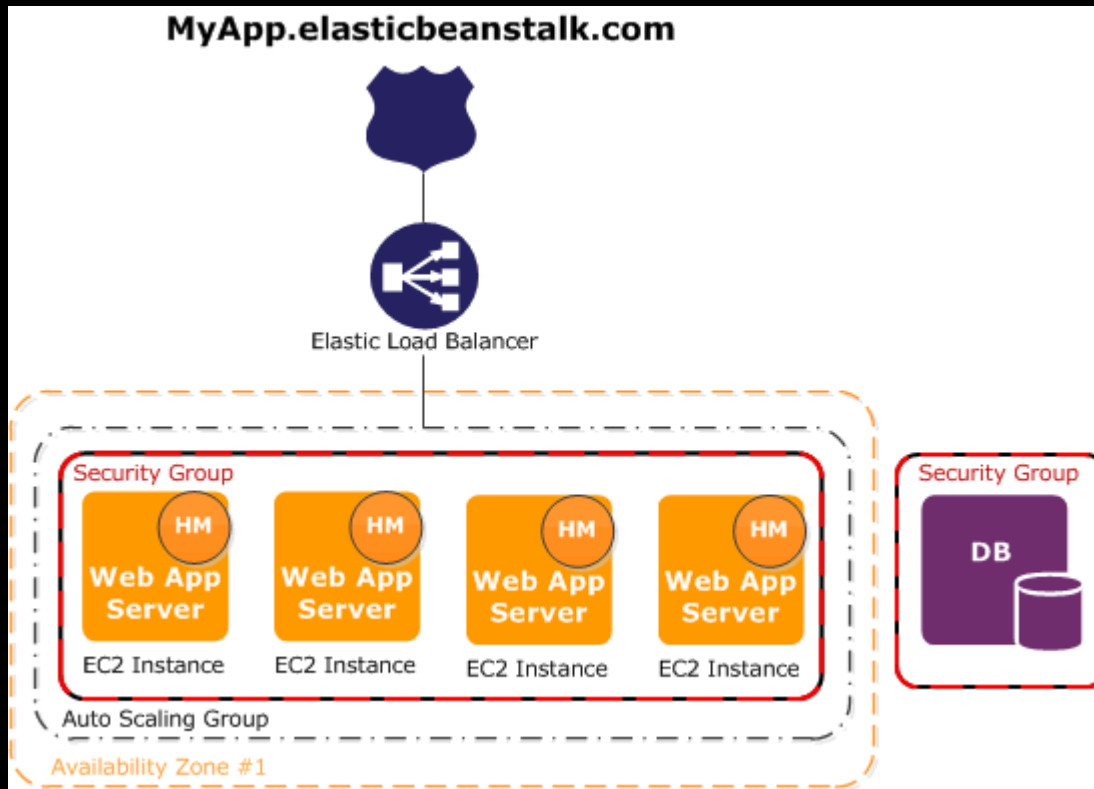
<http://www.appscale.com/>

# *Amazon Beanstalk*

- Amazon PaaS szolgáltatása 2011-től
- Támogatott nyelvek:
  - Python, Ruby, PHP, .NET, Java, Node.js
- Monitoring (CloudWatch)
- AutoScaling
- Load balancing
- Fejlesztő környezetek:
  - Management console, Git repo, IDE (Eclipse, Visual Studio)

<http://aws.amazon.com/elasticbeanstalk/>

# Amazon Beanstalk szerver

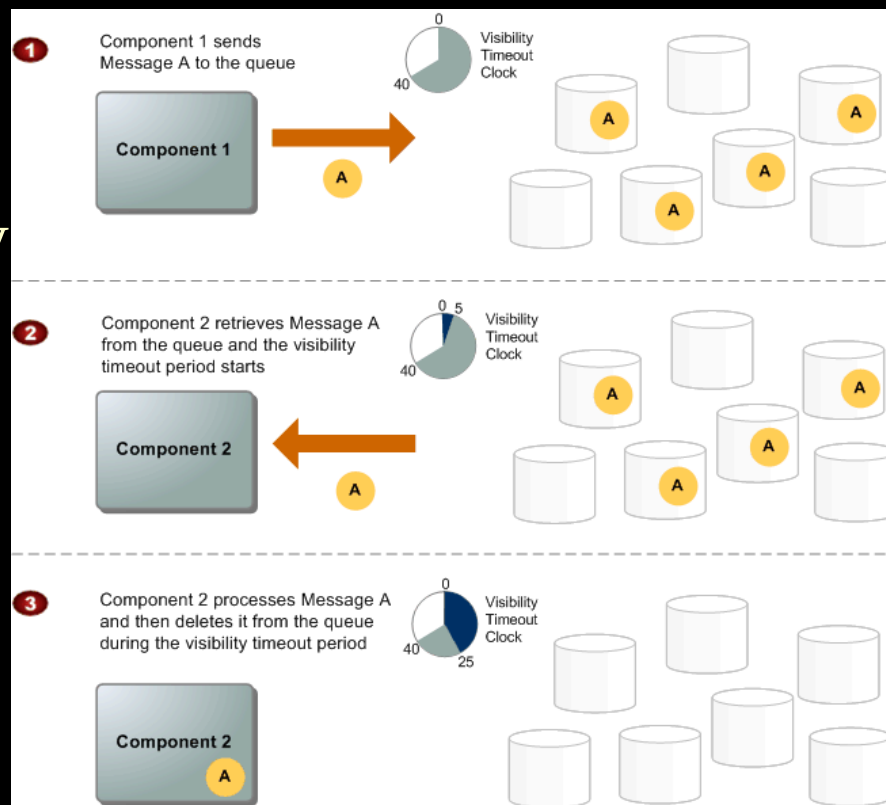


- Terhelés- kiegyenlített szerverpark
- Amazon Route53 (DNS)
- Nem lehet kinőni.

# Amazon SQS

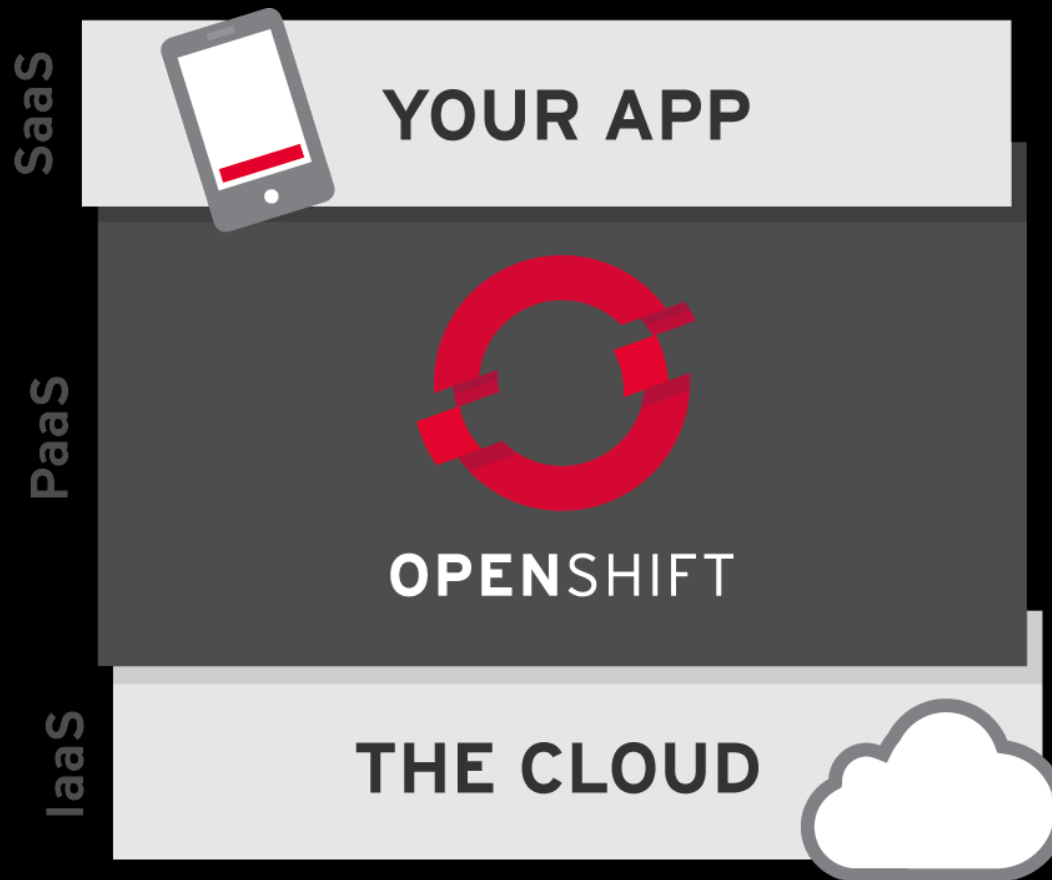
## SQS: Simple Queue Service

- Elosztott
- Redundáns
- At least-once-delivery
- Message sample
- Lifecycle
- Visibility timeout
- JMS kompatibilis



# OpenShift

- Open source PaaS (RedHat → IBM)

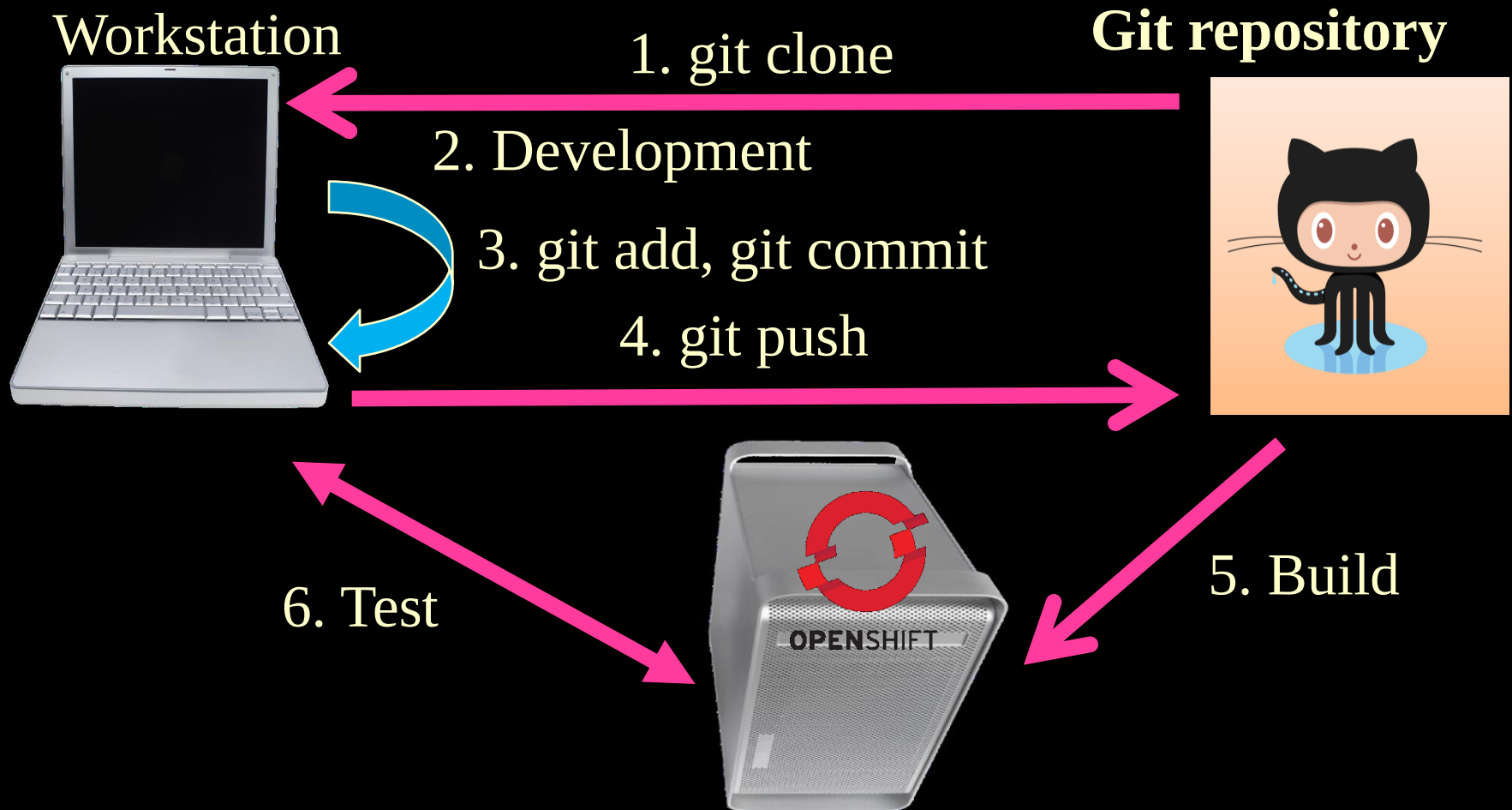


# OpenShift

- Red Hat PaaS szolgáltatása 2011-től
  - Red Hat felvásárlás: 2019. IBM
- Apache Licenc
- Alapvetően OpenStack felett fut, de más környezetekben is installálható
- Kubernetes-alapú környezet, vállalati policy-k
- Saját infrastruktúrán is telepíthető
- Támogatott nyelvek:
  - Haskell, Java, JavaScript, .NET, Perl, Python, Ruby
- Támogatott adatbázisok:
  - MS SQL, MongoDB, MySQL, PostgreSQL



# *OpenShift fejlesztési ciklus példa*



# Heroku

- Indulás: 2007, Salesforce, Ruby platform
- Ma: Támogatott nyelvek: Ruby, Java, Node.js, Python, PHP, Go, Scala, Clojure
- Dinók (dynos): konténer-alapú futtatási egységek
- Marketplace: több ezer add-on (DB, cache, monitoring, CI/CD)
- Adatbázis: Heroku Postgres, Redis
- Git-alapú deploy: `git push heroku main`
- Előny: nagyon gyors prototípus-fejlesztés, fejlesztőbarát
- Hátrány: költségesebb nagy terhelésnél, vendor lock-in

# *OpenShift vs. Heroku*

---

- OpenShift:
  - vállalati, Kubernetes-alapú, rugalmas, de komplex
  - nagyobb cégeknek.
- Heroku:
  - fejlesztőbarát, SaaS jellegű, gyors indulás
  - startupoknak és prototípusokhoz.

# *Alibaba Cloud EDAS*

- Enterprise Distributed Application Service (EDAS)
- Mikroszervizek, alkalmazásmenedzsment
- Fő funkciók: automatikus skálázás, terheléelosztás, szolgáltatás-regisztráció
- Integráció: Kubernetes, Spring Cloud, Dubbo
- Célcsoport: nagyvállalati alkalmazások, elosztott rendszerek
- Előny: szoros integráció az Alibaba Cloud ökoszisztémával
- Hátrány: erős vendor lock-in a kínai piacra

# *Számítási és tárolási felhők alapjai (4. ea)*

## *Elosztott fájlrendszerek*

*Szeberényi Imre, Ludmány Balázs*  
BME IIT

*<szebi@iit.bme.hu>*



# *Tartalom*



- Bevezetés
- AFS (történelem, architektúra)
- Lustre
- Ceph
- ZFS
- GlusterFS
- CERN rendszerek (CernVMFS, EOS)
- Összefoglalás / IO-500

# *Fájlrendszerek*

CÉL: Adatok véletlen-szerű strukturált elérése

- bájt, rekord, adatbázis szinten
- hibatűrő módon
- Blokkos elérésű eszközökön megvalósított fájlrendszerek
  - Lokális
    - speciális hardver (szalag, CD/DVD, flash, SSD, ...)
  - Hálózaton keresztül
    - (meg)osztott
    - Elosztott
- Speciális adatok elérése (procfs, configfs, ...)

# *Osztott fájlrendszerek*

- Blokkszintű hálózati eléréssel rendelkező eszközök (SAN) fölött működő kliensek biztosítják a hozzáférést és a kölcsönös kizárást.
  - CXFS (SGI)
  - Veritas Cluster FS
  - Nasan FS (DataPlow)
  - General Parallel FS (IBM)
  - Cluster Volumes (MS CSV)
  - Global FS (RedHat)
  - QFS (SUN)
  - VMFS (VMware)
  - Xsan (Apple)

[http://en.wikipedia.org/wiki/List\\_of\\_file\\_systems](http://en.wikipedia.org/wiki/List_of_file_systems)



# *Elosztott fájlrendszerek*

- Nagyméretű klaszterek
- Földrajzilag is elosztott rendszerek
- Heterogén környezet
- Hibatűrés, replikáció, migráció
  - FAL (DEC, 1970)
  - NFS (Sun, 1985)
  - AFS, CODA, InterMezzo
  - Lustre, SFS
  - GFS (Google)
  - GlusterFS (RedHat)
  - OCFS (Oracle, GNU)
  - Hadoop HDFS
  - BeeGFS (Fraunhofer)
  - Ceph (RedHat, SUSE)
  - Gfarm file System
  - GPFS (IBM)
  - OrangeFS (Parallel Virtual FS)
  - CernVMFS (Cern)
  - S3 (Amazon)
  - OneFS (EMC)
  - Moose FS (Core Tech)
  - DFS (MS)
  - ObjectiveFS (Objective Sec. Group)
  - CERN-EOS
  - Quobyte

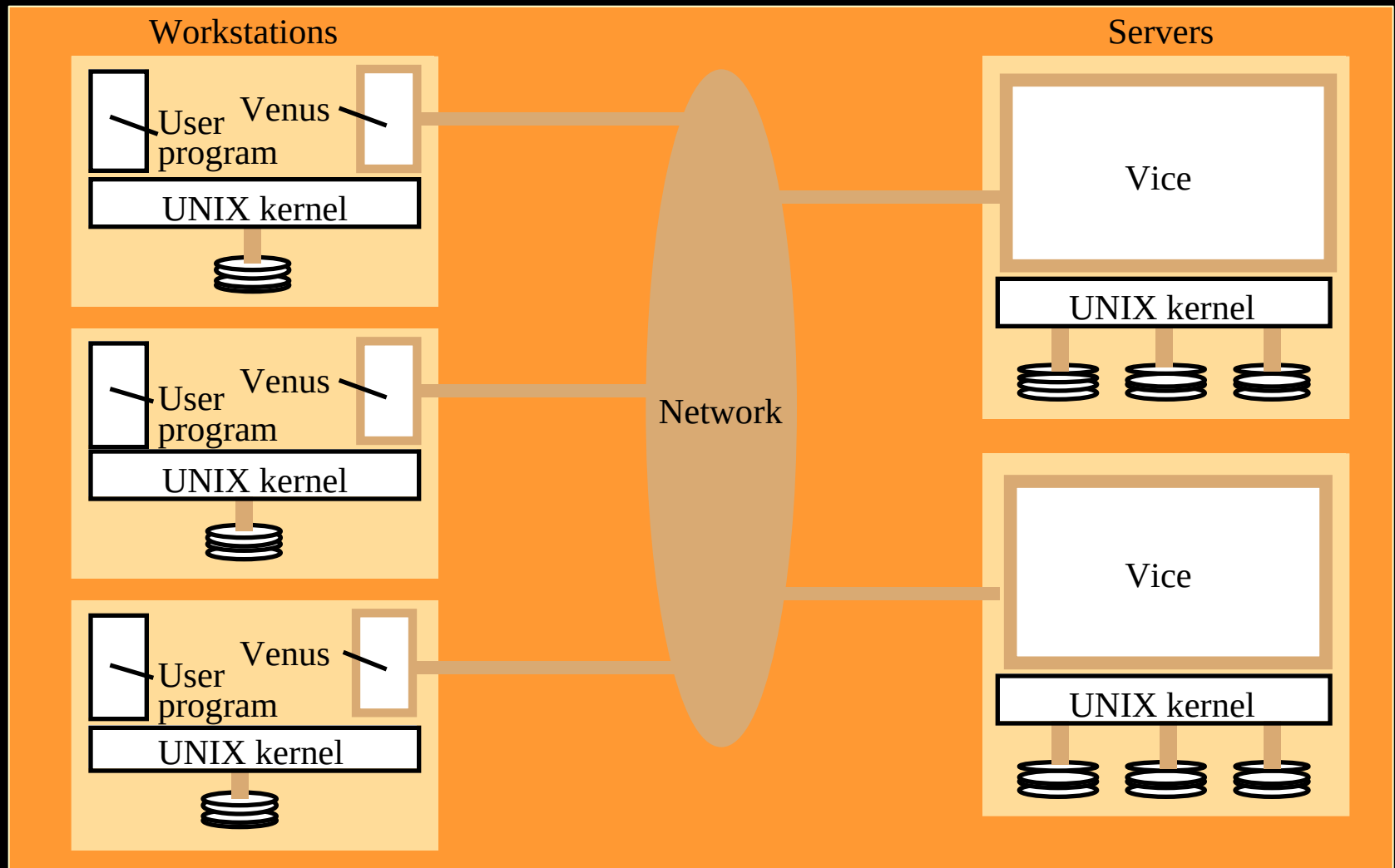
# *AFS (Andrew File System)*

- Elosztott fájlrendszer, ami fájlok megosztására alkalmas lokális és távolsági hálózaton.
- Transzparens fájlhozzáférést biztosít.
- Az NFS-hez hasonló, annak alternatívájaként jött létre.
- Ma az OpenAFS számos UNIX, LINUX, WinX platformon elérhető.
- CERN-ben ma is használják, bár 2020 óta kivezetés alatt van (helyette EOS), de sokszor jobban teljesít az öreg vasakon.

# *AFS történelem*

- Carnegie Mellon Egyetemen 1984-ben fejlesztették ki UNIX környezetben. Ma azonban nem csak UNIX változat létezik.
- A fő cél az volt, hogy az egyetemi korlátozott sávszélességű hálózaton hatékony fájllelérést tegyenek lehetővé.

# *AFS processzek*



# *AFS alapfogalmai*

---

- Cellák
- Kötetek
- Tokenek
- Cache menedzser
- Fájl védelem
- Fájl névtér

# *AFS cella*

- Egy AFS cella alá azok a szerverek tartoznak, melyek adminisztrációja közös, és az AFS felé egyetlen közös fájlrendszert alkotnak.
- Tipikusan az egy domain név alá tartozó gépek egy AFS cellát alkotnak.
- Általában a domain név valamilyen változata a cellanév.
- A munkaállomások a felhasználókról a cella szervertől kérnek információkat.

# *Kötetek*

- A diszkterületet az AFS további részekre, osztja ezek az AFS kötetek.
- Az AFS kötet egy tárolóegység ami a fájlok és katalógusok adatait tárolja.
- Az AFS kötetek fájlok formájában jelennek meg a befogadó operációs rendszerben, így azok könnyen átmozgathatók, akár másik gépre is.

# Tokenek

- **Az AFS** nem használja a UNIX felhasználói azonosítóját (UID). Ha ezt tenné, akkor minden UNIX gépen azonos UID kiosztásnak kellene lennie, mint az NFS-nél.
- Az azonosításhoz AFS tokenet alkalmaznak, ami egy egyedi azonosítást tesz lehetővé.
- Egy token adott ideig (24 óra) érvényes.



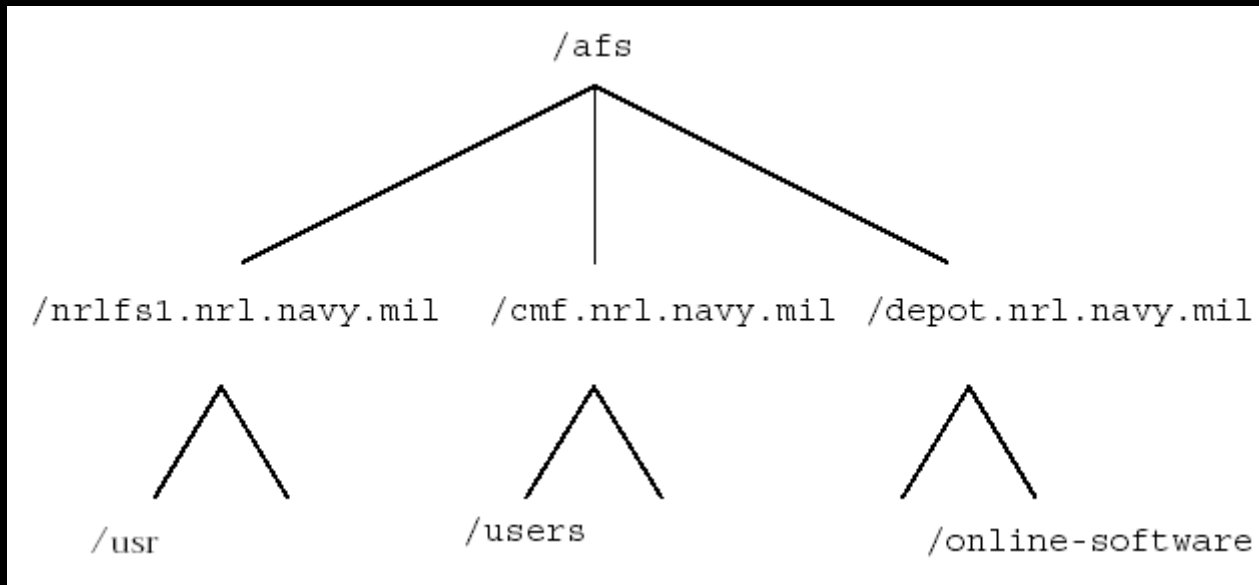
# Cache menedzser

- A korlátozott sávszélesség miatt a működés központi eleme a cache, ahova az éppen használt fájlok letöltődnek.
- A cache menedzser feladata a cache-ben tárolt információk frissítése, karbantartása.
- Amennyiben a cache-ben tárolt fájlrészlet változik, úgy azt vissza kell tölteni a szerverre.
- Ha a szerveren változik meg a fájl, akkor arról Callback technikával értesít minden cache-t.

# Védelem

- A védelmi mechanizmus némileg eltér az alap UNIX védelmi rendszertől.
- A UNIX 3x3-as védelmétől pontosabban szabályozható ACL (Access Control List) segítségével.
  - Lookup (l)
  - Insert (i)
  - Delete (d)
  - Administer (a)
  - Read (r)
  - Write (w)
  - Lock (k)

# Névtér



# Névtér /2

- UNIX-hoz hasonló hierarchikus struktúra
- Az AFS gyökér névtér rendszerint a /afs. Az alatta levő szinteket a cellák képviselik.
  - adminisztratív domain
    - AFS szerverek halmaza egy cégnél, egyetemen, laborban stb.
  - Lokális cella
    - alapértelmezett cella, amihez az adott munkaállomás csatlakozik.
  - idegen cella
    - más cella az AFS névtérben

# *Venus és Vice*

---

- Venus
  - AFS kliens által futtatott processz.
- Vice
  - AFS szerver által futtatott processz.

# *Fájl műveletek*

- A kliens munkaállomás a szerverrel csak az open/close műveletek kiszolgálásakor kommunikál.
- A fájl megnyitásakor a Venus a teljes fájlt a cache-be tölti, és a fájl lezárásakor írja azt vissza.
- Az adatok olvasását/írását a lokális másolaton a kernel végzi.
- A Venus a katalógusokat és a szimbólikus linkeket is a lokális gyorsítótárban tárolja.
- A fenti gyorsítótárazási mechanizmus alól a katalógusok módosítása a kivétel, aminek a végrehajtásáért a közvetlenül szerver a felelős.

# *Fájl megosztás*

- Lokális fájlokhoz hasonlóan.
  - nincs külön mount
  - nem kell belépni a másik gépre
  - csak jogosultság kell
- A /afs katalógus alatt tetszőleges cella fájljai elérhetők.
  - Természetesen megfelelő jogosultsággal.
  - Csak a megfelelő útnevet kell hozzá tudni.
- A fájlmegosztást nem korlátozza a földrajzi távolság, vagy az adott operációsrendszer típusa.

# *Login és autentikáció*

1. Bejelentkezéssel együtt token is generálódik
2. Külön kell token generálni.
  - klog,

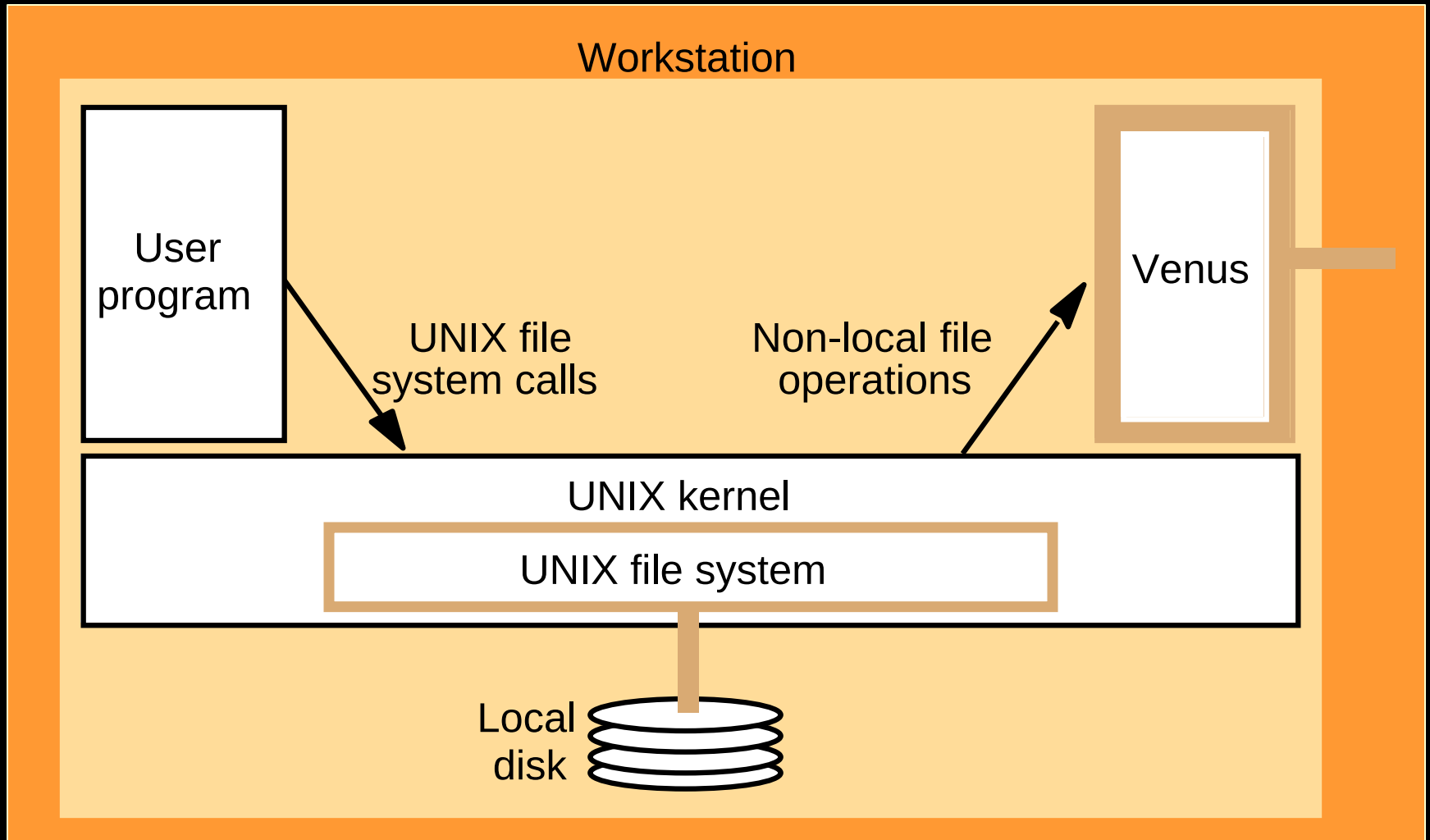
```
Tokens held by the Cache Manager:
--End of list--
szebi:$ /usr/afs/bin/klog
Password:
szebi:$ /usr/afs/bin/tokens
Tokens held by the Cache Manager:
User's tokens for afs@bme.hu [Expires Apr  7 00:47]
--End of list--
.....
User's tokens for afs@cern.ch [Expires Apr  7 00:53]
User's tokens for afs@bme.hu [Expires Apr  7 00:47]
```



# *Megvalósítás*

- A kliens oldali programok a szokásos módon, rendszerhívással kezelik az állományokat.
- A távoli fájlok megnyitásakor Venus processzhez jut a kérés, amit az lebont az útnév alapján.
- Az alacsonyszintű I/O kezelését a befogadó operációs rendszer végzi. A gyorsítótár a lokális gép diszkjén jön létre.

# *Rendszerhívás szint*



# *AFS parancsok*

## **Az AFS parancsok 3 csoportba oszthatók:**

- Fájlszerver parancsok (fs)
  - AFS szerver információk listázása
- Védelmi parancsok(pts)
  - ACL listák létrehozása
- Authentikációs parancsok
  - klog, unlog, kpasswd, tokens

# *AFS előnyei*

- Gyorsítótárazásból fakadó előnyök:
  - Lényegesen csökkenti a hálózati forgalmat.
  - Alacsonyabb sávszélességnél is jól használható.
- Helyfüggetlenség:
  - Az AFS a földrajzi helyet a szerver oldalon rendeli fájlnevhez. Így a névtér helyfüggetlen.
- Skálázhatóság:
  - A rendszer tervezési fázisában igen nagyra (~10000 kliens) tervezték. A kliens/szerver arányt pedig 200:1-re. Mindkét értéket túlteljesíti.

# *AFS előnyei /2*

- Single systems image (SSI):
  - Egy fájlserver kialakítása lényegesen egyszerűbb, mint NFS-sel.
- Fokozott biztonság:
  - Kerberos használata
  - ACL használata
- Fájlok egyszerű megosztása
- Egyszerű rendszer menedzsment
- Robosztus
- Replika lehetőség.

# *AFS hátrányai*



- Minden munkaállomásra installálni kell.
- Háttérszerver komplexitása.
- Tokenek érvényességének lejártából fakadó gondok.

# *Lustre*

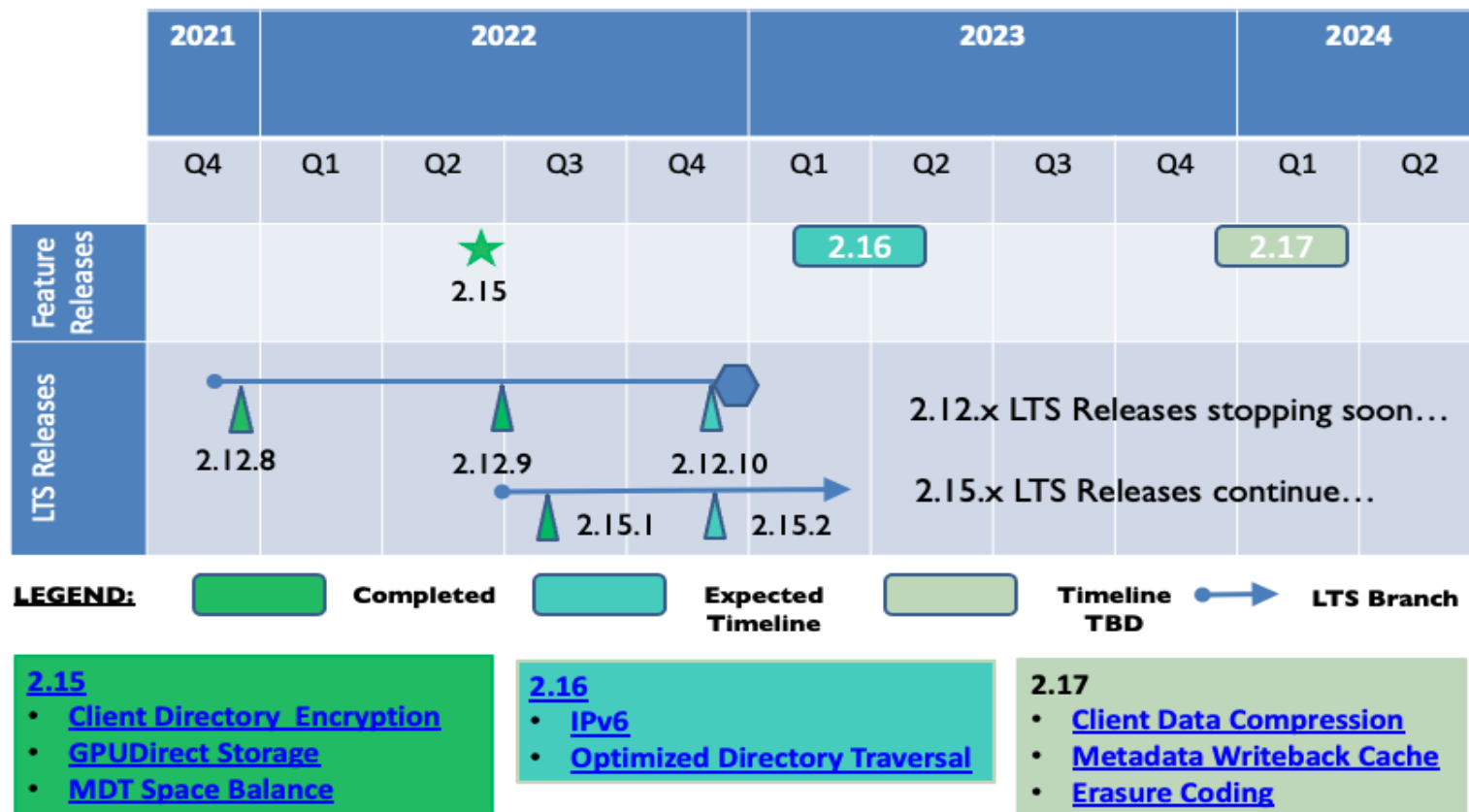
- Objektum-orientált elosztott fájlrendszer.
- Jól skálázható.
- Nagyméretű klaszterekhez, és nagy fájlokhoz tervezték.
- Lustre 2007-től GPL.
- SUN ZFs
- top 30 szupergépből 15 Lustre fájlrendszert használ.

# *Lustre történelem*

- 1999 by Carnegie Mellon University
- Lustre 1.0 2003-ban (Cluster File Systems)
- 2007-ben SUN felvásárolta a CFS-t.
  - Open source software (RedHat, SUSE, ...)
- 2010-ben Oracle felvásárolta az SUN-t
  - 2011-ben 1.8 supportot megszüntette (számos szervezet folytatta)
  - Whamcloud, **OpenSFS**, EOFS,
- 2012-ben Whamcloud-ot megvette az INTEL
- 2019- OpenSFS és EOFS visszaszerzi a nevet
- 2022: Lustre 2.15
- 2024: Lustre 2.16



# Lustre Community Roadmap



\* Estimates are not commitments and are provided for informational purposes only

\* Fuller details of features in development are available at <http://wiki.lustre.org/Projects>

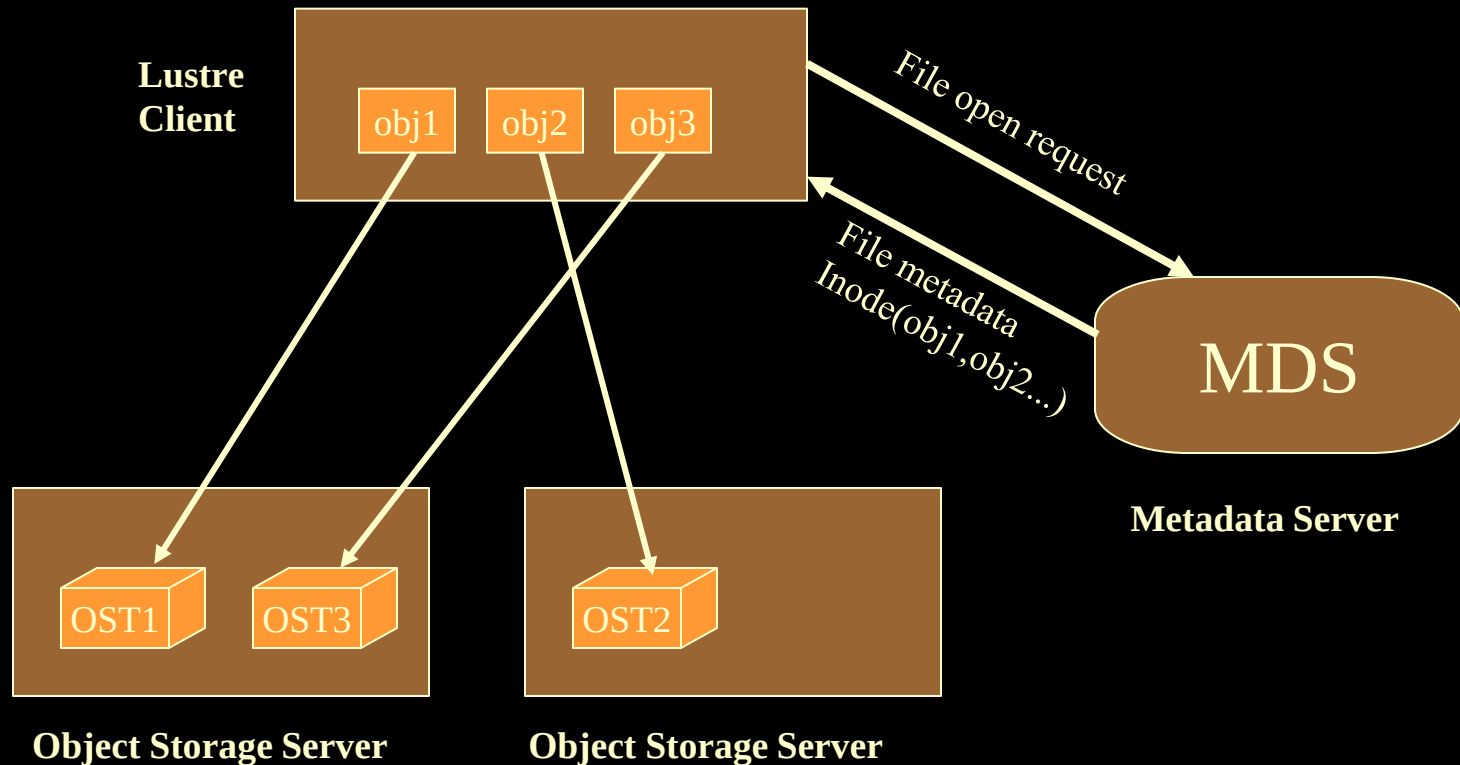


# *Lustre architektúra*



- Három fő funkcionális egysége van:
- Metadata szerver (MDS), ami a fájl neveket, katalógusokat, védelmi kódokat és egyéb metaadatot tárol.
- Object storage szerverek (OSS), melyek az adatokat tárolják.
- Kliens ami az adatokat felhasználja, létrehozza.

# *Lustre architektúra*



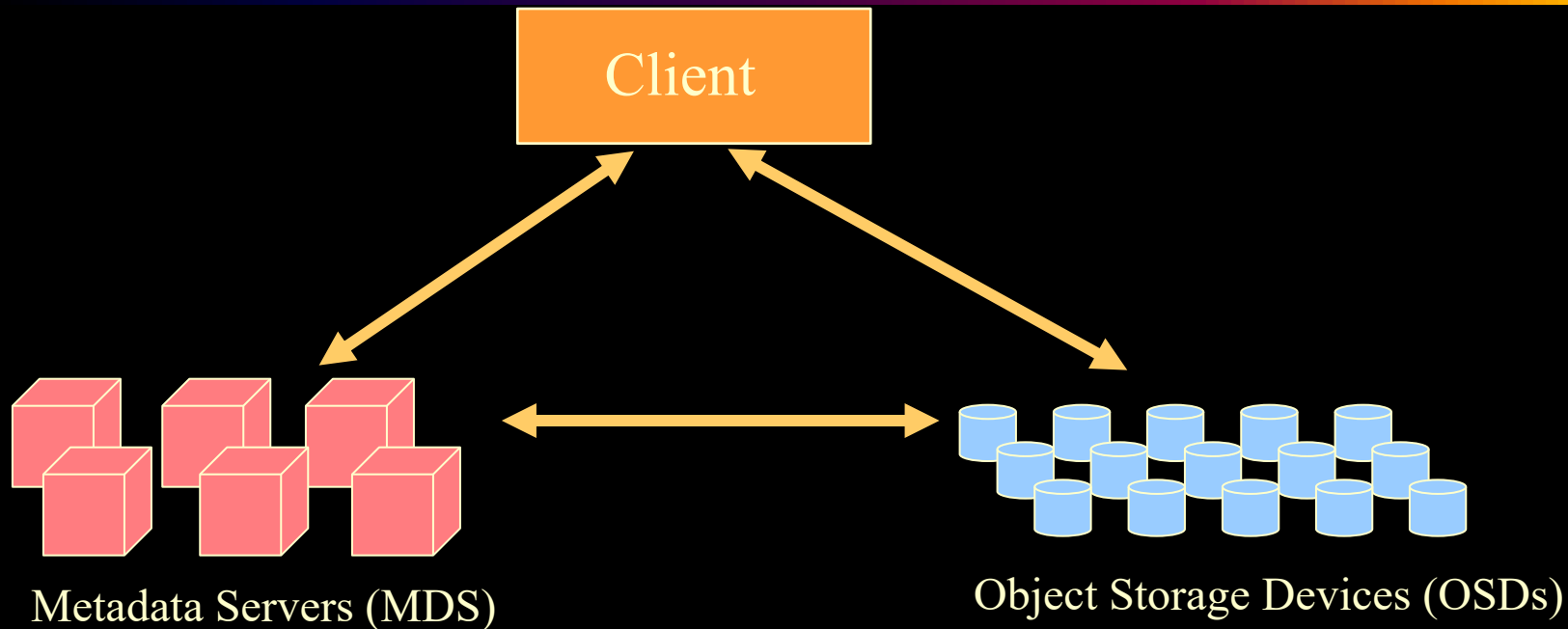
# *Lustre architektúra /2*

- Az adatok logikai kötetmenedzsmenttel ellátott RAID tárolókban tárolódnak, amit az OSS és az MDS dedikált módon használ.
- Jelenleg egy módosított ext4 fájlrendszer a logikai tároló. ZFS support
- Amikor egy kliens fájlt akar elérni, először az MDS-ben meg kell keresnie.

# *Lustre architektúra /2*

- A fájl egyes darabjai több OSS-en tárolódhatnak, ami a kliens és az OSS között szűk keresztmetszet kialakulását gátolja.
- A kliensek nem módosítják közvetlenül az OSS-ben tárolt adatokat, hanem ezt a OSS-re bízzák, szemben a GFS megoldásával.
- Ez a módszer növeli a megbízhatóságot és a hibatűrést.

# Ceph



Ötlet azonos: Metadata és az adatok szétválasztása.

2007: doktori disszertáció, 2010 Linux, 2012: Inktank

2014: RedHat, 2015: Ceph CE

2023: Ceph Reef

2024: Squid

# *Ceph jellemzői*

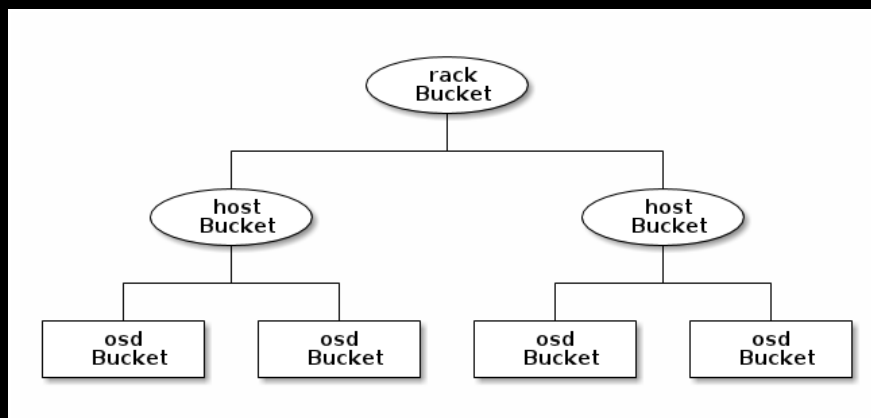
---

1. Adat és metaadat teljes szeparációja
  - Objektum alapú tároló
  - Független metaadat tárolás
  - CRUSH – adatelosztási funkció
2. Intelligens diszkek (RADOS)
  - Megbízható autonóm elosztott objektum tároló
3. Dinamikus metaadat menedzsment
  - Adaptív és skálázható

# CRUSH

## Inode helyett CRUSH

- Teljesen leírja az adatok elhelyezkedését
- Kiszámítható az objektumok elhelyezkedése
- Elosztott, hierarchikus
- Nincs allokációs lista
- Kis helyen tárolható

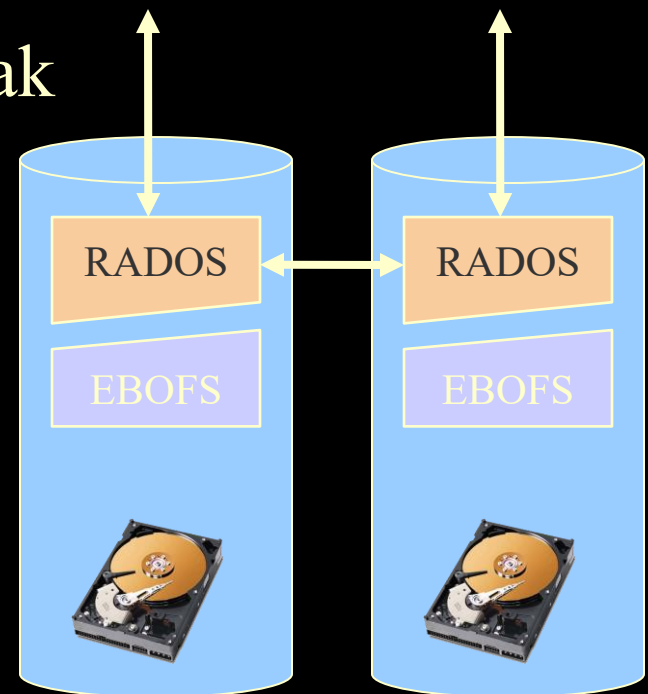




# RADOS

## RADOS: Reliable Autonomic Distributed Object Store

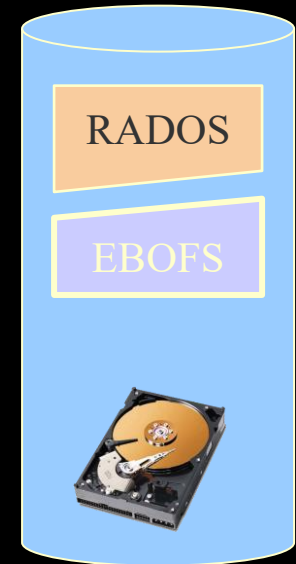
- Intelligens OSD
  - OSD-k páronként kommunikálnak
- OSD-k egy logikai tárolót alkotnak
  - megbízhatóak
  - önmenedzselők
  - elosztottak
- EBOFS a lokális tárolásért felel



# *Alacsony-szintű tárolás*

## **EBOFS: Extent and B-tree-based Object File System**

- copy-on-write technika alkalmazása
  - Egyszerű visszalépni egy előző konzisztens állapotba
- User-space implementáció
  - Nem használ VFS-t



# ZFS

- Sun: 2001-2004, 2005-től Solaris része
- 2010-ben Oracle felvásárolja a Sun-t
- Zettabyte File System
- 128 Bit - extra nagy kapacitás
- Pool elvű tárolók – elosztott sávszélesség és kapacitás
- Tranzakció kezelés – Copy on Write
- Snapshots (ro) és klónozás
- Adat integritás – ellenőrző összeg (külön)

# *ZFS kapacitások*

- $1 \text{ ZB} = 10^{21}$        $1 \text{ ZiB (zebi B)} = 2^{70}$
- $2^{64}$  snapshot
- $2^{48}$  fájl / dir
- $2^{64}$  byte / fájl
- $2^{78}$  byte / pool
- $2^{64}$  device / pool
- $2^{64}$  pool / system

# *Hogyan kapunk diszk címet*

Hagyományos FS esetén:

- FS(1): filename  $\rightarrow$  object (inode)
- FS(2): object  $\rightarrow$  volume LBA
- VM: volume LBA  $\rightarrow$  array LBA
- RAID: array LBA  $\rightarrow$  disk LBA

Sok réteg, szigorú szeparáció, eltérő gyártók

# *Hogyan kapunk diszk címet (2)*

ZFS esetén:

- ZPL: filename  $\rightarrow$  object
- DMU: object  $\rightarrow$  DVA
- SPA: DVA  $\rightarrow$  LBA

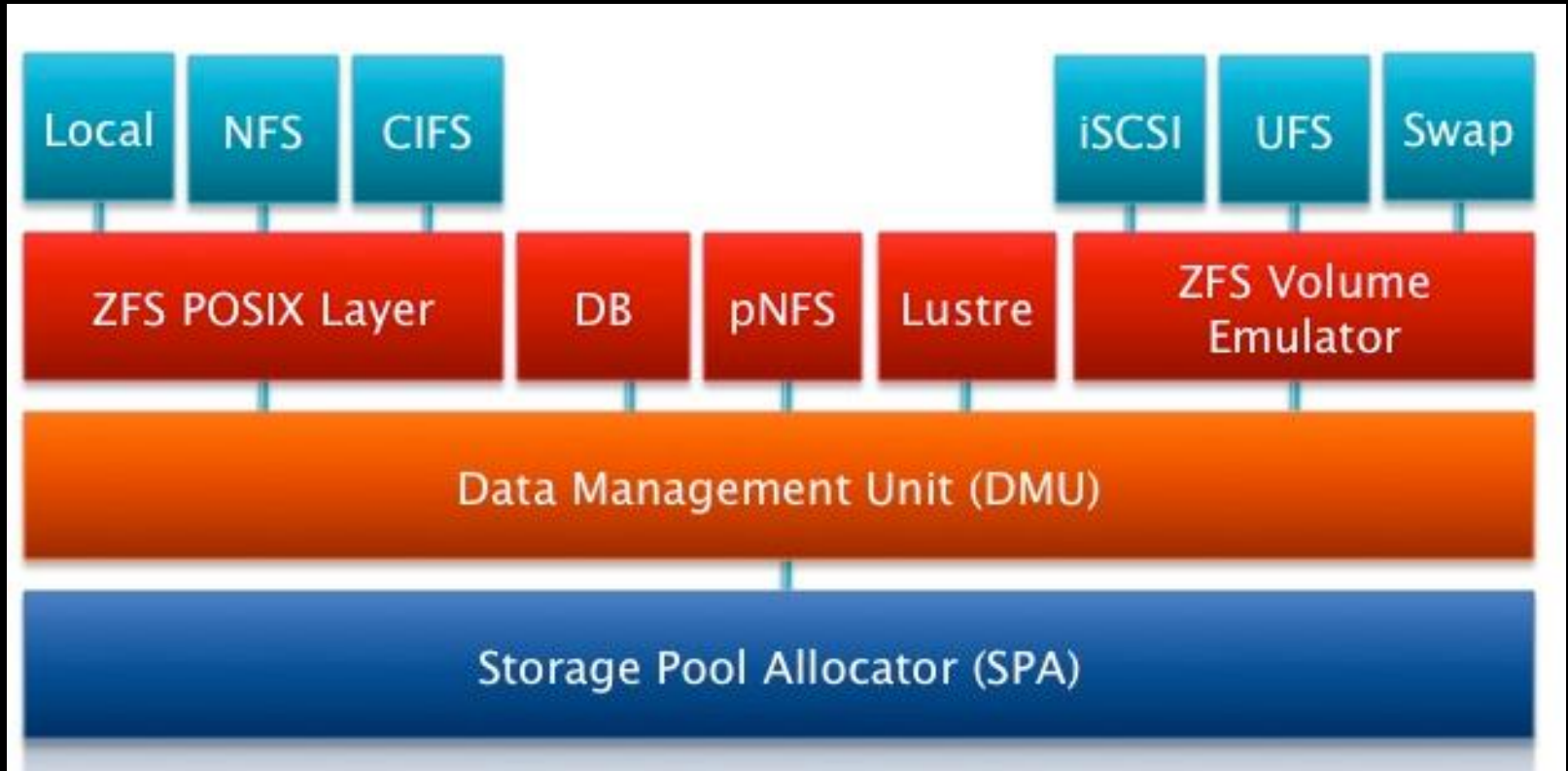
ZPL: ZFS POSIX layer (standard syscall)

DMU: Data Management Unit (transactional object store)

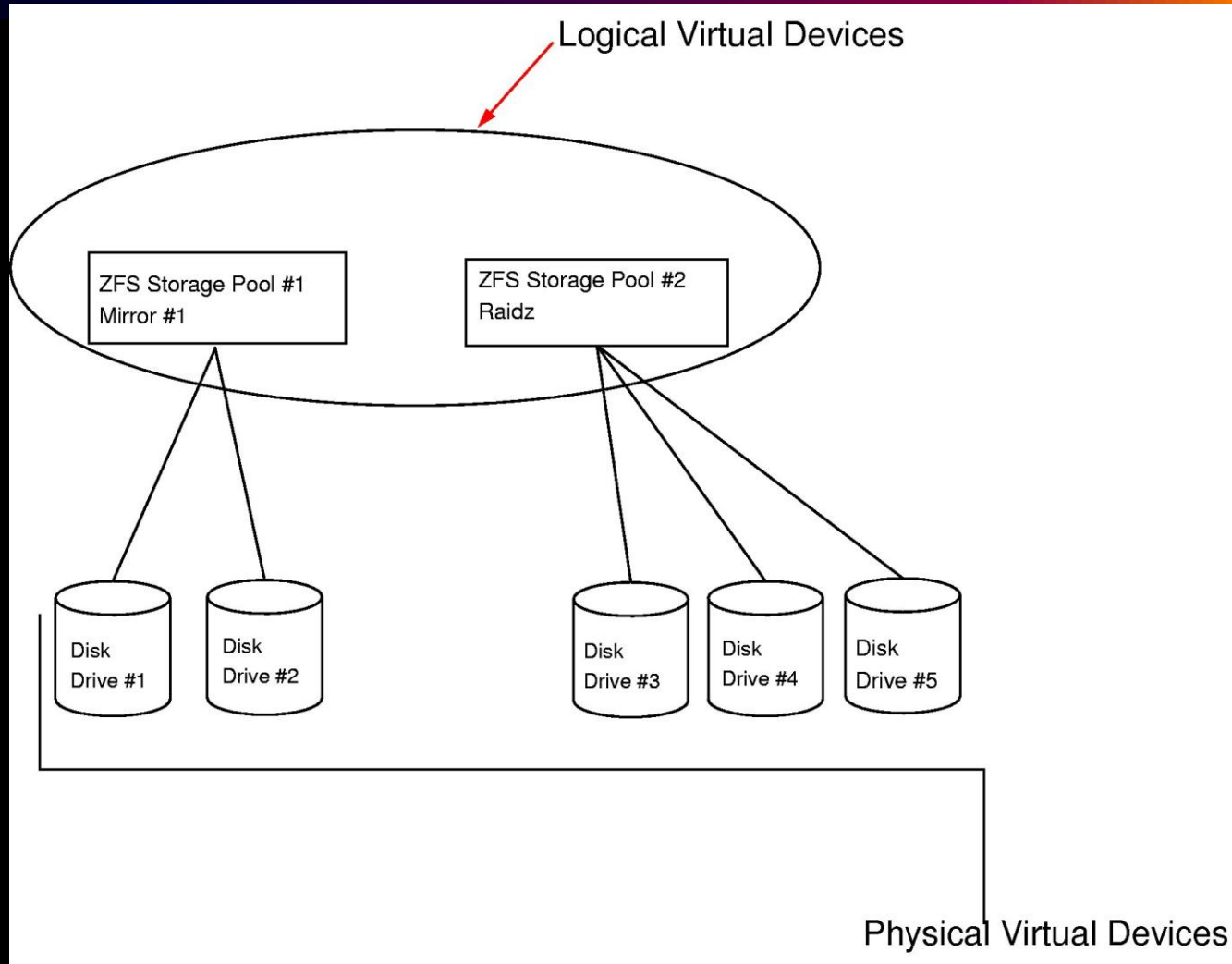
DVA: Data Virtual Address (vdev + offset)

SPA: Storage Pool Allocator (block alloc, data transform)

# ZFS Architektúra



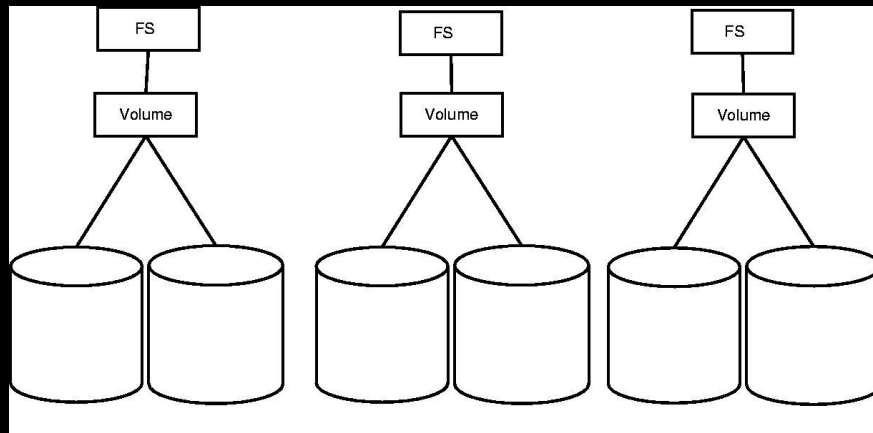
# ZFS – VM hasonlóság



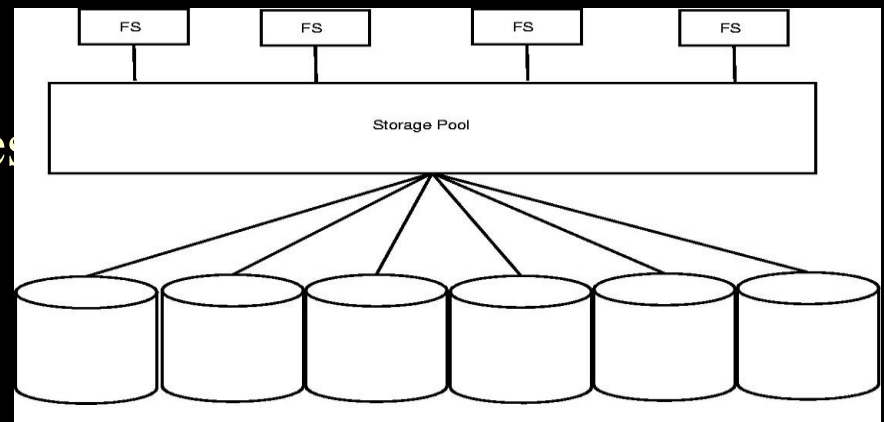


# Kötet és Pool

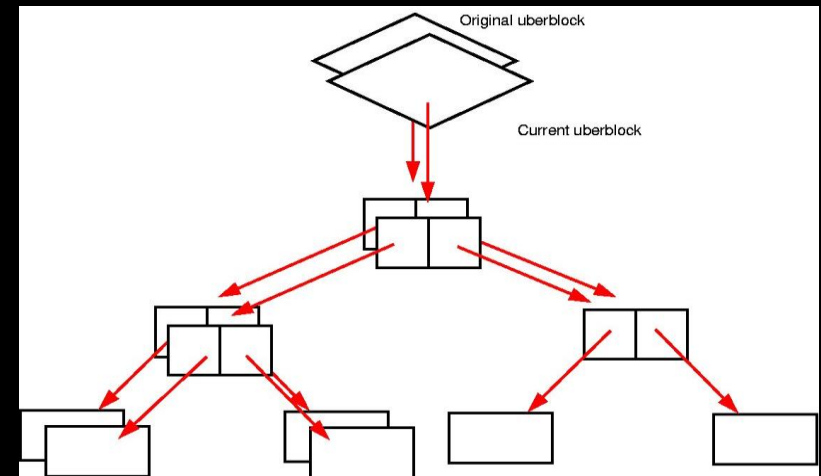
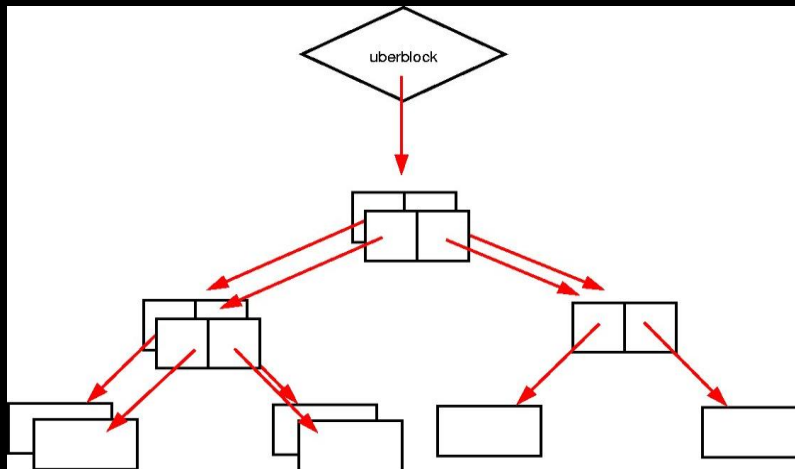
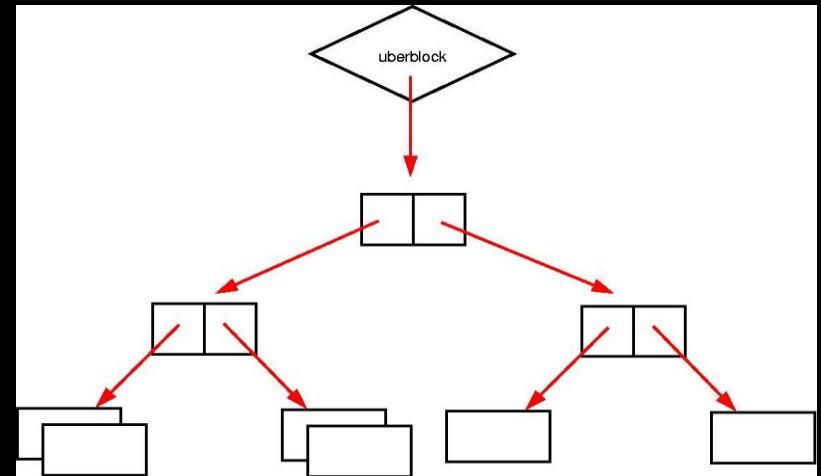
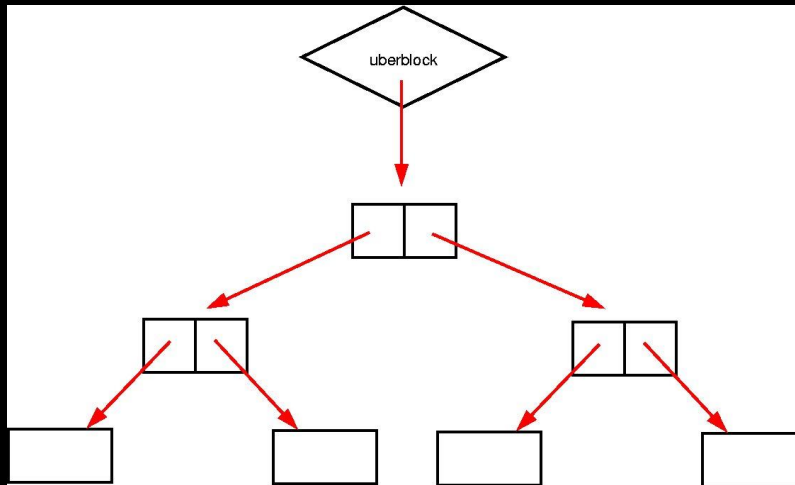
- Hagyományos kötet kezelés



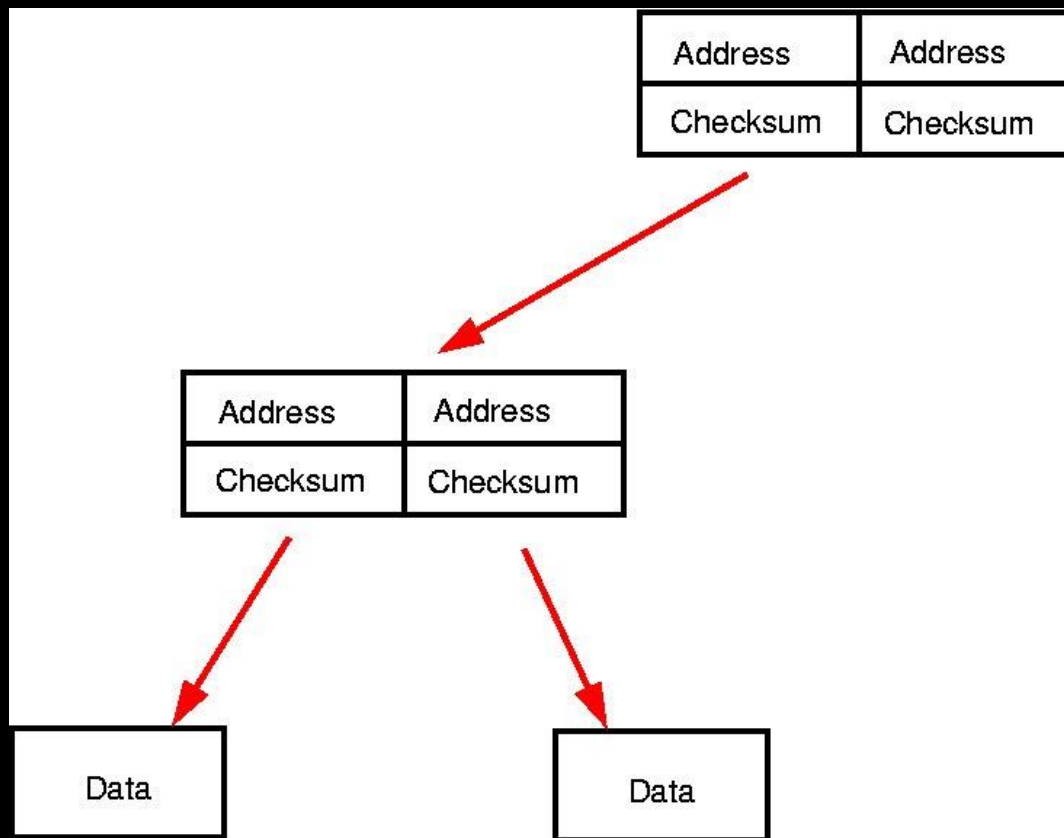
- Pool:
  - Automatikus méretezés
  - osztott sávszélesség



# ZFS - Copy on Write (COW)



# ZFS – ellenőrző összeg



# *ZFS elérhetősége*

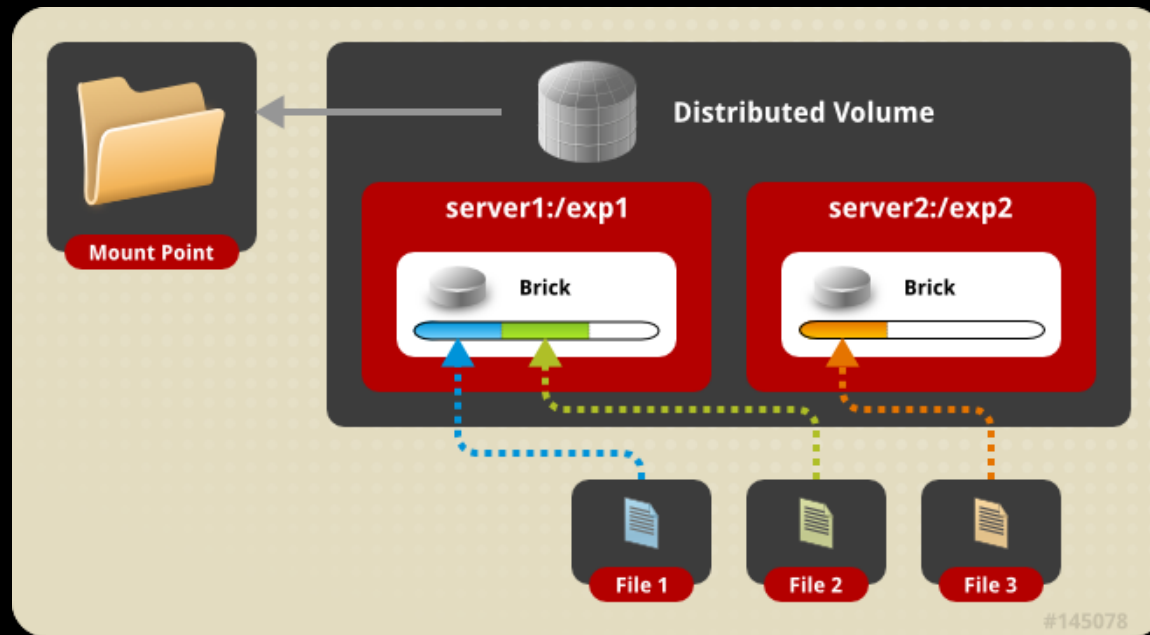
- OpenSolaris, OpenIndiana
- BSD, OSX
- Linux: CCDDL és a GPL üti egymást
- OpenZFS – 2013-tól (Linux 6.x)
- Linux FUSE
- Native ZFS (Gentoo, Ubuntu, LLNL)

<http://en.wikipedia.org/wiki/ZFS>

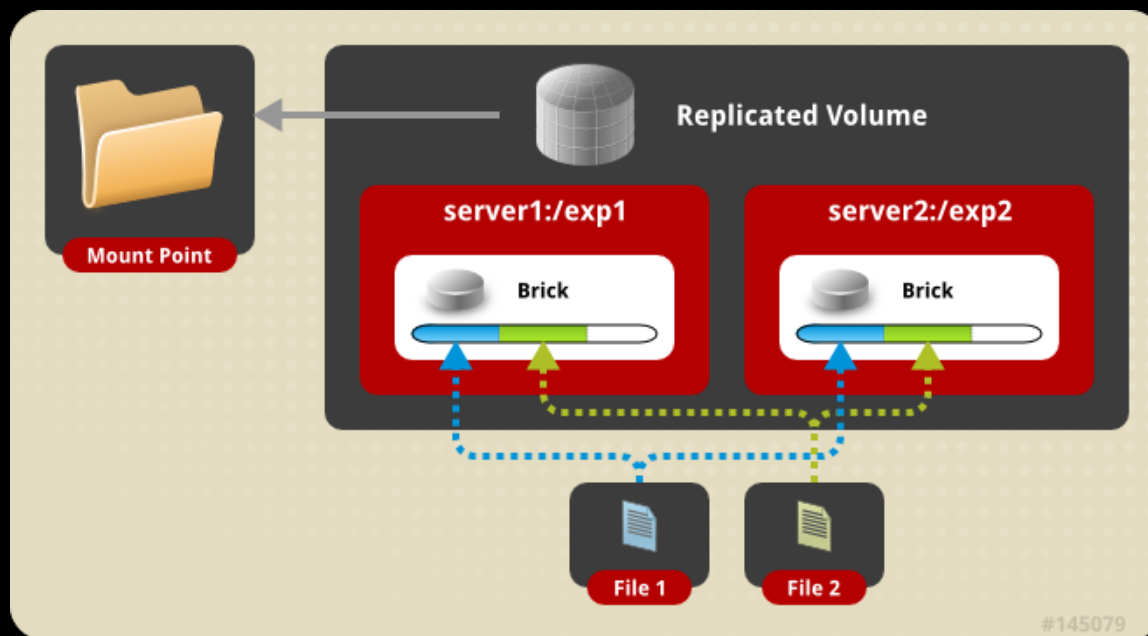
# GlusterFS

- Célkitűzés FUSE alapokon megvalósítani elosztott fájlrendszert.
- A céget 2011-ben megvette a RedHat, minek hatására a közösség láthatóan hanyatlásnak indult.
- 2013-ban újra megindult a fejlesztés
- 2015-től RH nagyon tolja
- 2019-ben IBM felvásárolja a RH-t
- Legutolsó verzió: 11.x (2024)

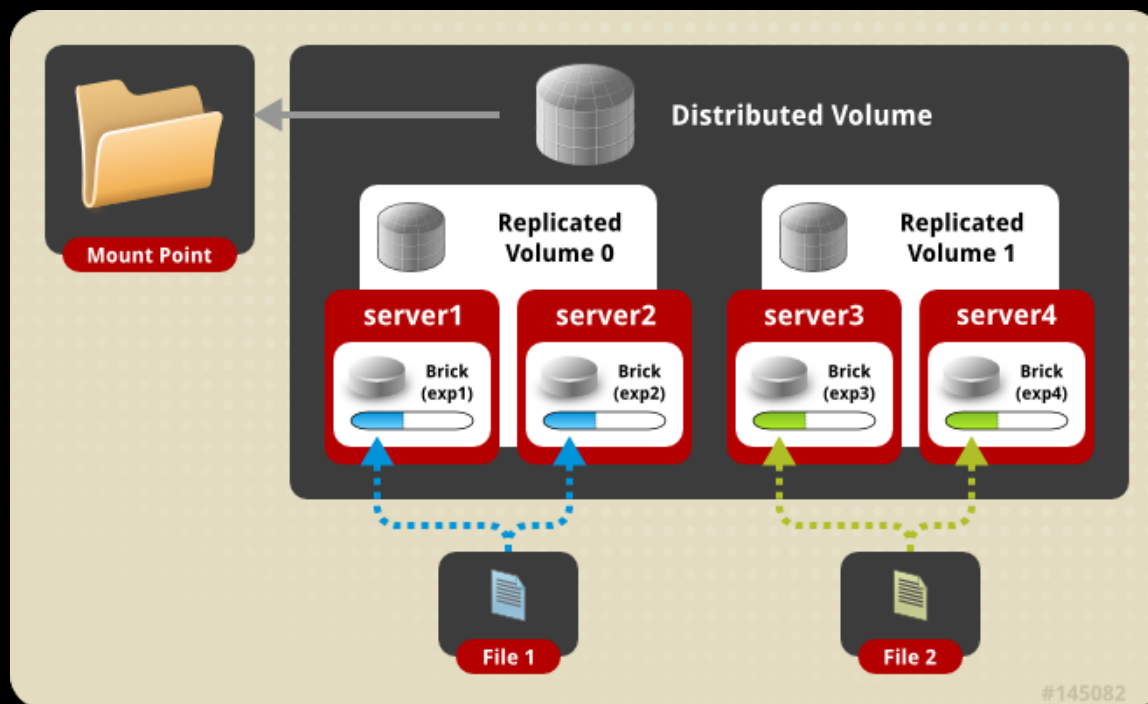
# GlusterFS – Distributed Vol.



# GlusterFS – Replicated Vol.

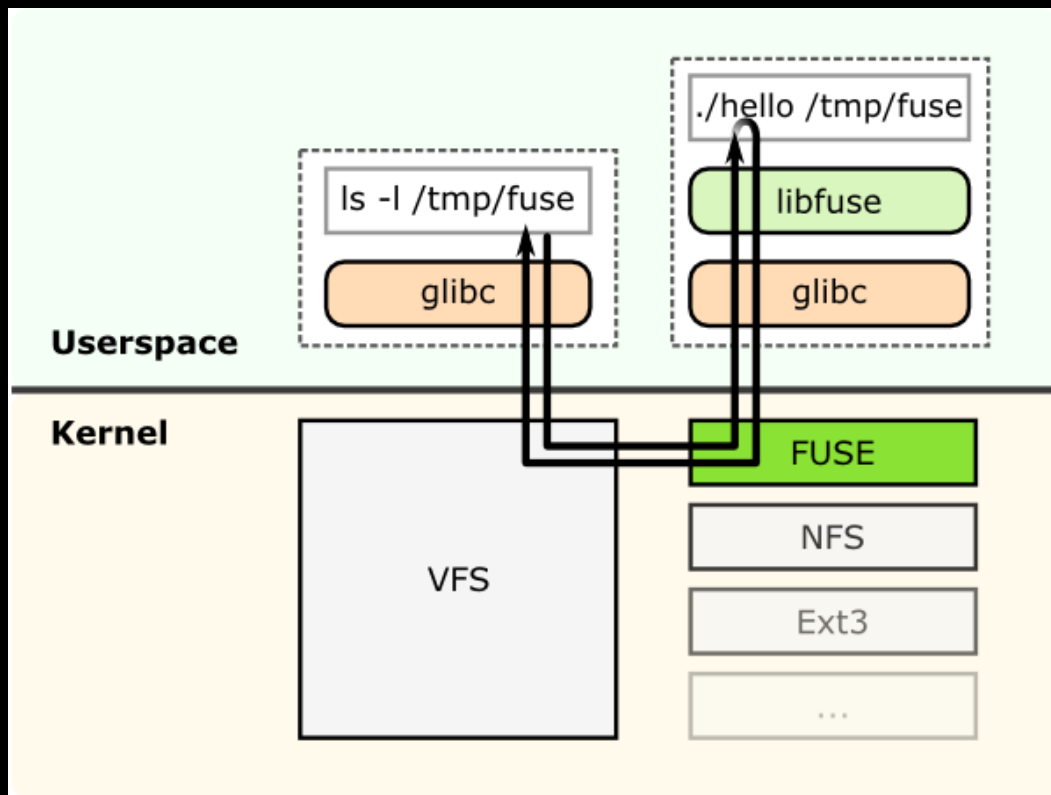


# *Distributed Replicated Vol.*

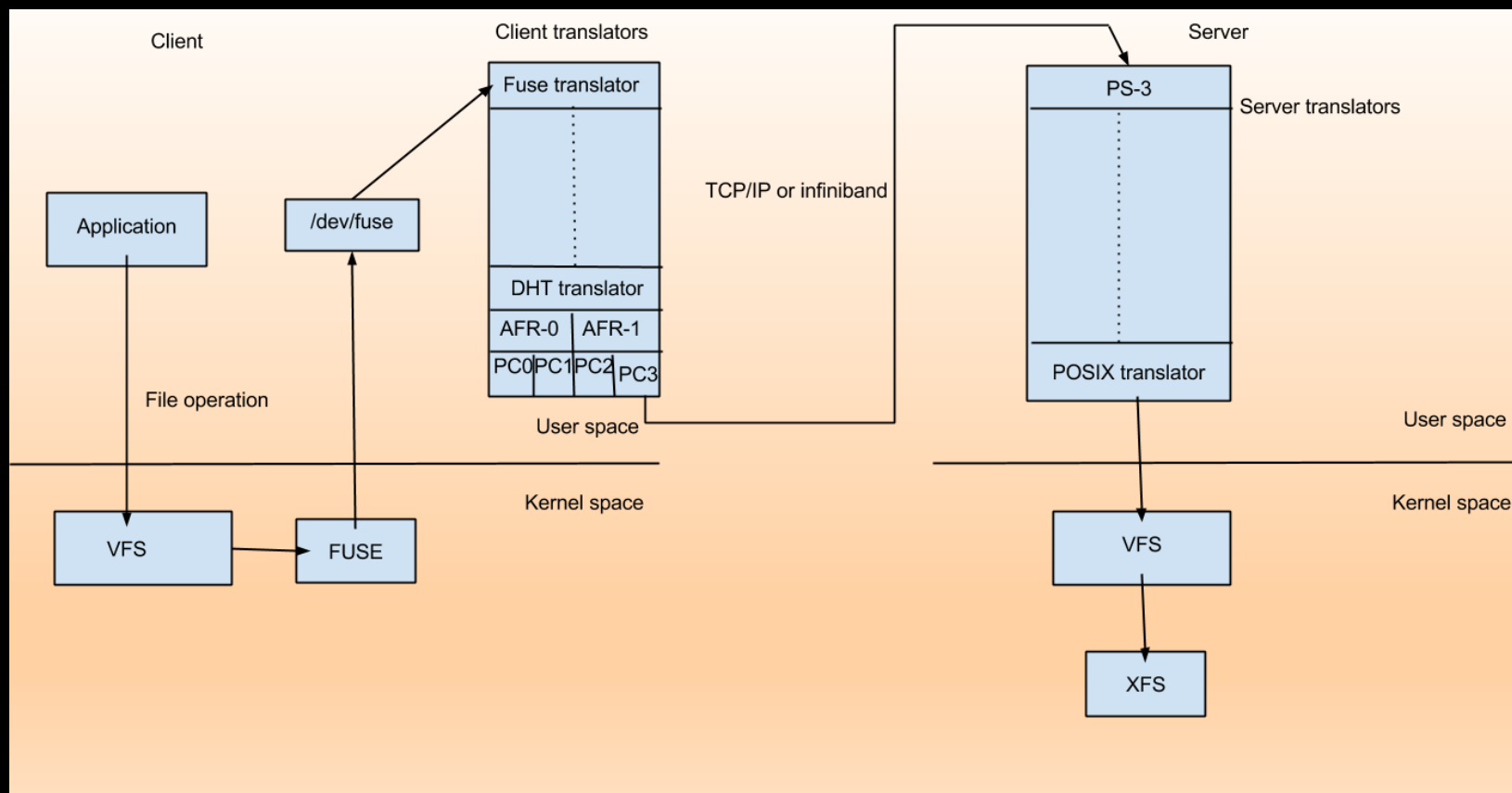




# FUSE



# Működés



# *CernVM-FS*

- Cél: Programok és kísérleti adatok (LHC) elérésének biztosítása a kutatók számára.
- Tartalom alapú tároló
- HTTP → tűzfalak átengedik
- Adatok integritásának ellenőzése hash-ekkel.

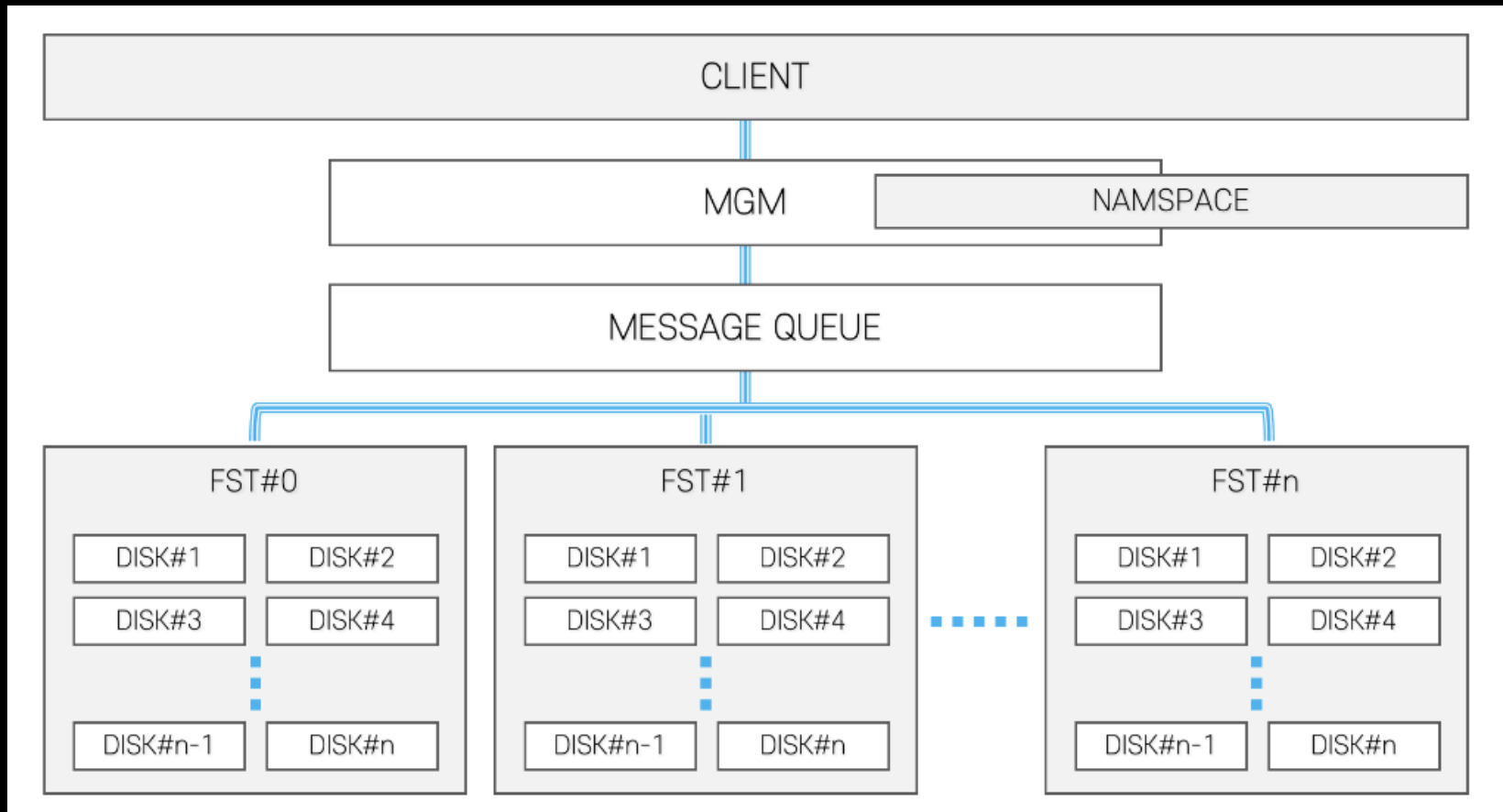
# *CernVM-FS /2*

- POSIX read-only fájlrendszerként van megvalósítva (FUSE)
- HTTP cache
- Adatok és metaadatok átvitele on demand módon.
- Főleg SL (Scientific-Linux) disztribúciókhoz

# *EOS – CERN fájlrendszere*

- Nagy teljesítményű, skálázható elosztott fájlrendszer.
- Nagy I/O teljesítmény petabájtos és exabájtos adatmennyiségekhez.
- Rugalmas, többprotokollos hozzáférés (POSIX, HTTP, WebDAV, XRootD)
- Magas rendelkezésre állás és hibatűrés.
- Skálázhatóság több adatközpont között.
- CERN SSO, X.509, Kerberos, LDAP
- OpenSource

# Cern-EOS Architektúra



# AFS → EOS

## Fő örökségek az AFS-ből az EOS-ban:

- Globális namespace – /afs/... → /eos/... Logika
- ACL alapú jogosultságkezelés → ACL + AuthZ plugin rendszer
- Replikációs elv – volume replication → namespace-level replica sets
- Szervezeti modell – cellák → EOS 'space' és 'instance' fogalmak

## Különbségek:

- Teljesen új technológiai alap: Redis / QuarkDB metaadat, XRootD / HTTP hozzáférés
- Nincs AFS-kliens cache, hanem szerveroldali caching és multi-protocol access
- Modern REST és POSIX interfészek, multi-site architektúra

# Quobyte

The logo consists of a horizontal bar with a color gradient from dark blue on the left to bright yellow on the right.

- Diszk-bázisú objektum orientált tároló
- Skálázható
- Redundáns (0. v. 3 replika)
- HPC környezet
- HA és FT
- Multi-tenant



# *Melyiket hol használják?*

- Szuperszámítógépek, nagy felhők
  - Lustre, Sepectrum Scale, BeeGFS, WekaFS, DAOS
- Department/divízió
  - NFS, SMB/CIFS, AFS (legacy), GlusterFS, CephFS
- Wokrgroup
  - SMB, NFS, Lokális NAS
- Teljesítmények összehasonlítása
  - IO-500 lista + közösség

# *Top HPC fájlrendszerek*

1. Frontier (OLCF, USA) – Lustre – Exascale rendszer, Cray Slingshot interconnect
2. Alps (CSCS, Svájc) – BeeGFS – GPU-optimalizált, DGX SuperPOD architektúra
3. Fugaku (RIKEN, Japán) – Fujitsu FEFS – NVMe-alapú, sűrű IO optimalizálás
4. LUMI (CSC, Finnország) – Lustre – EuroHPC infrastruktúra, ZFS backend
5. JUPITER Booster (FZJ, Németország) – IBM Spectrum Scale – Multi-tier caching, magas sávszélesség
6. Perlmutter (NERSC, USA) – Lustre + all-flash tier – GPU-rétegzett HPC architektúra

# *Top HPC fájlrendszerek*

7. ThetaGPU (ALCF, USA) – Ceph + Lustre hybrid – Ceph alapú objektumtároló integráció
8. Marconi100 (CINECA, Olaszország) – Spectrum Scale – IBM Power9 környezetre hangolt
9. Selene (NVIDIA, USA) – WekaFS – AI/HPC hibrid rendszer, extrém alacsony latency
10. CERN-EOS (CERN, Svájc) – EOS – Kutatási adatok POSIX + HTTP hozzáféréssel

Forrás: [io500.org/list](https://io500.org/list) (2025. június)

# *IO-500.org alapján*

| Fájlrendszer                       | Top 15-ben | Top 500-ban | Megjegyzés                                        |
|------------------------------------|------------|-------------|---------------------------------------------------|
| DDN EXAScaler / Lustre             | 7          | 200         | Exascale HPC-rendszerek fő fájlrendszere          |
| DAOS (Intel)                       | 4          | 20          | Új generációs, objektumalapú, NVM-optimalizált FS |
| WekaIO / WekaFS                    | 2          | 10          | AI-orientált, NVMe-alapú FS                       |
| Nyílt Lustre (community)           | 2          | 200         | Klasszikus open-source Lustre                     |
| IBM Spectrum Scale / Storage Scale | 0          | 60          | Széles körben használt,                           |
| BeeGFS                             | 0          | 40          | Közepes méretű GPU-klaszterekben                  |
| Ceph                               | 0          | 8           | Kiegészítő objektumtárolóként                     |
| EOS (CERN)                         | 0          | 5           | Tudományos adatarchívumokban                      |
| Egyéb (pl. Panasas, xFusion)       | 0          | 10          | Kis mintában, speciális OEM-megoldások            |

# Trendek

- Lustre továbbra is domináns a HPC klaszterekben.
- BeeGFS gyorsan terjed GPU-orientált rendszereknél.
- Spectrum Scale (GPFS) erős vállalati környezetekben.
- Ceph megjelent mint hibrid objektumtároló (ThetaGPU).
- CERN-EOS továbbra is meghatározó az európai tudományos adatinfrastruktúrában.